Design of AI Systems – Module 8 - Group 22

Erik Andreasson (MPDSC) - 200310304498 - eriub@chalmers.se,
William Norland (MPDSC) - 200111085155 - williamnorland@gmail.com

2026-03-22

We the students hereby declare that we have both actively participated in solving every exercise. All solutions are entirely our own work, without having taken part of other solutions.

Hours spent

- Erik Andreasson: 34
- William Norland: 30

Erik Andreasson (MPDSC) - 200310304498 - eriub@chalmers.se,
William Norland (MPDSC) - 200111085155 - williamnorland@gmail.com

Project Introduction

1.1 Background & Topic

Natural Language Processing (NLP) is a subfield of machine learning concerned with enabling computers to understand and work with human language. Despite its central importance to modern AI systems, NLP has not been covered in depth in this course, which was a primary motivation for us to propose this project. Our goal being to use the mini-project as an opportunity to explore this area of AI systems and build intuition for how computers can represent the semantic meaning of words.

A fundamental challenge in NLP is the representation of meaning from textual content. How is the meaning of a word encoded in a form that a machine learning model can work with? The simple approach is one-hot encoding, where each word in a vocabulary of size V is assigned a unique vector in $\mathbb{R}^V$ with a single element set to 1. While simple, this representation has every word equidistant from every other word meaning that the encoding of "king" is no more similar to "queen" than it is to "bicycle". One-hot encoding does not allow for any representation of similarity in meaning or context between words.

Word2Vec, introduced by Mikolov et al. at Google in 2013, addressed this by learning dense, low-dimensional word representations called embeddings directly from raw text. The core insight is the distributional hypothesis being that words that appear in similar contexts tend to have similar meanings. By training a shallow neural network to predict a word from its surrounding context words (or vice versa), the internal weight matrix of the network converges to a representation where semantically related words occupy nearby regions of the embedding space. The result is that geometric relationships in this space encode meaningful structure with the classic example being that the vector arithmetic $\text{king} - \text{man} + \text{woman}$ produces a vector close to queen.

1.2 Objectives

The concrete objectives of this project are:

- Implement the full Word2Vec pipeline from scratch, using PyTorch only for tensor operations. All layers, training logic, and optimization are implemented manually.
- Train and compare two output objectives: full softmax and negative sampling, analyzing both their mathematical formulations and their practical computational trade-offs.
- Study how the volume of training data affects the quality of learned embeddings by training across three corpus sizes.
- Visualize the learned embedding spaces using PCA and UMAP, and evaluate embedding quality qualitatively through nearest-neighbor analysis.

1.3 Relation to Course Concepts

Although Word2Vec originates in NLP, it is related to the latent factor models studied in this course in the context of recommendation systems. In collaborative filtering, the goal is to factorize a sparse user-item interaction matrix $\boldsymbol{R} \approx \boldsymbol{P}\boldsymbol{Q}^T$ into a matrix of latent user factors $\boldsymbol{P}$ and latent item factors $\boldsymbol{Q}$. The core idea being that the latent factors are learned representations that capture the underlying structure of user preferences. This can then be used to make inference on unseen data to for example, recommend movies to users.

Word2Vec can be understood as the same idea applied to language. The interaction matrix is represented via the input embedding matrix $\boldsymbol{W}_{\text{in}}$ and output embedding matrix $\boldsymbol{W}_{\text{out}}$ as the two latent factor matrices. Training pushes words that co-occur frequently into compatible regions of the

latent space similarly to how collaborative filtering pushes users and items with high interaction into compatible latent factor directions.

This project extends beyond simply applying this concept by implementing the full optimization machinery by hand without the use of a packaged library. The forward and backward passes through each layer, the mini-batch gradient descent, and the loss evaluations are all done from scratch. This gives us direct experience in the details of how the embedded representations are actually formed. This enables us to directly understand the impact of certain design choices and hyperparamater values beyond input and output from a completed PyTorch module.

Implementation Methodology

As part of the challenge to our project we decided to implement nearly all aspects of the model from scratch. This includes the full stack of modules and components required to preprocess the data, feed the data to the model, each neural network layer, and train the final result. Our main methodology for the implementation of each component was to focus on creating an inherited structure focused on separation of concerns, reusability, and scalability. Especially for the neural network layers we wanted forward and backward passes to operate over each layer independently of type or internal structure. This would allow us to create multiple models to experiment with.

2.1 Tokenizer

The implementation of our tokenizer converts an input corpus to a vocabulary where words are indexed by a number which represents their tokenized value. These can be used for referencing and dereferencing of words from the training set as well as constructing one-hot encoded vectors. Furthermore, the tokenizer exposes a method for constructing continuous bag of words (CBOW) context and target word pairs for training. The method uses a sliding window over the dataset of sentences, constructing a pair of specified k number of context words along with the missing target word that the model is to predict.

The tokenizer uses a preprocessing method when constructing the vocabulary which allows us to collect as much useful information as possible. Considering that our goal is to analyze the semantic meaning of individual words and not sentences, we made some specific design choices in the preprocessing. Both of our operations were chosen on the basis that our representations do not attempt to represent sentiment, inflection, grammatical usage, or any other contextualized differences between word representations. Instead, we focus on representing and comparing the words in isolation. Therefore, we perform the following operations:

- Remove punctuation: The inclusion of punctuation would be misleading since it is a tool for expressing meaning in the context of a sentence. For instance a question, a pause, an exclamation, and so on. They are grammatical tools which do not alter the meaning of the word they are attached to. For instance, “apple!”, “apple?”, “apple,”, and “apple.” all have the word apple which we aim to represent, not the inflection of the word.
- Convert to lowercase: This keeps all words equal before their representation, removing possible differences in representation for capitalized words such as “Apple” and “apple”.

2.2 Dataloader

The dataloader is responsible for feeding batched data to the model during each epoch. Two implementations were developed to address different memory and performance constraints.

The base DataLoader class operates over a pre-allocated list of all training context-target pairs. It supports optional shuffling via index permutation and yields batches of a fixed size by slicing this

list on each iteration. While straightforward, this approach requires all context-target pairs to be held in memory simultaneously which becomes impractical as corpus size grows, since the number of CBOW pairs scales with both the number of sentences and the window size. Having a maximum number of sentences larger than 200 while one-hot encoding was enabled as output often crashed our computers from running out of memory.

To address this, StreamingDataLoader was introduced. Rather than storing all pairs upfront, it accepts a generator function that produces context-target pairs on demand. The total pair count is determined at construction time by exhausting the generator once without retaining any pairs, keeping peak memory at the size of a single sample regardless of corpus size. Pairs are accumulated into batches incrementally and yielded as they fill, with any remaining pairs at the end of a dataset sweep yielded as a final partial batch.

The downside is that StreamingDataLoader does not support shuffling since pairs are always yielded in corpus order with no index structure to permute. For the purposes of this project this was deemed acceptable, since the sliding window provides variation in the input and ordered context-target pairs should not considerably affect the final result.

All training runs in this project use StreamingDataLoader, with a separate generator call for the training split (sentences 0–80%) and validation split (80–100%), controlled by the start and end fraction parameters exposed by the tokenizer's generate_CBOW_pairs method.

2.3 Layers

We chose to define our layers in an object oriented way. The reason for this being that we aim to increase the readability of our code by relating python objects to mathematical objects. Hopefully, creating a successful bridge between the theory and the implementation.

Each layer of our model can be modeled as a function.

$$\hat{\boldsymbol{y}} = f(\boldsymbol{v})$$

This function can contain any arbitrary operation on the input vector given that it returns a Tensor object as a result. This effectively functions as an interface for clients to use.

2.3.a Softmax Activation

Central to the straightforward implementation of the model is the softmax activation function which allows the final linear layer to output a vector of predicted probabilities for the target word across the entire tokenizer vocabulary. This is done by turning a vector of numbers $\boldsymbol{z} \in \boldsymbol{R}^n$ in to a vector of probabilities $\hat{\boldsymbol{y}}$ s.t $\sum_i \hat{y}_i = 1$. The softmax function is defined as:

$$\hat{\boldsymbol{y}}_i = \frac{e^{z_i}}{\sum_{j=1}^{n} e^{z_j}}$$

The gradients are calculated automatically during the backward pass and are set as follows:

$$\delta_{ij} = \begin{cases} 1 \text{ if } i = j \\ 0 \text{ if } i \neq j \end{cases}$$

$$\frac{\partial \hat{y}_i}{\partial z_j} = \hat{y}_i(\delta_{ij} - \hat{y}_j)$$

$$\text{diag}(\hat{\boldsymbol{y}}) = \begin{pmatrix} \hat{y}_1 & 0 & \dots \\ 0 & \hat{y}_2 & \dots \\ \vdots & \vdots & \ddots \end{pmatrix} \quad \hat{\boldsymbol{y}}\hat{\boldsymbol{y}}^T = \begin{pmatrix} \hat{y}_1^2 & \hat{y}_1\hat{y}_2 & \dots \\ \hat{y}_2\hat{y}_1 & \hat{y}_2^2 & \dots \\ \vdots & \vdots & \ddots \end{pmatrix}$$

$$\frac{\partial \hat{\boldsymbol{y}}}{\partial \boldsymbol{z}} = \text{diag}(\hat{\boldsymbol{y}}) - \hat{\boldsymbol{y}}\hat{\boldsymbol{y}}^T$$

The single element version $\frac{\partial \hat{y}_i}{\partial z_j}$ of the derivative is included for clarity as it makes it easier to understand the full vector ($\hat{\boldsymbol{y}}$).

In order to calculate the partial derivative of the loss given the input $\boldsymbol{z}$ to the activation layer we can apply the upstream partial derivative denoted $\frac{\partial L}{\partial \hat{\boldsymbol{y}}}$ using the chain rule:

$$\frac{\partial L}{\partial \boldsymbol{z}} = \left(\frac{\partial \hat{\boldsymbol{y}}}{\partial \boldsymbol{z}}\right)^T \cdot \frac{\partial L}{\partial \hat{\boldsymbol{y}}}$$

$$\frac{\partial L}{\partial \boldsymbol{z}} = (\text{diag}(\hat{\boldsymbol{y}}) - \hat{\boldsymbol{y}}\hat{\boldsymbol{y}}^T) \cdot \frac{\partial L}{\partial \hat{\boldsymbol{y}}}$$

$$\frac{\partial L}{\partial \boldsymbol{z}} = \text{diag}(\hat{\boldsymbol{y}}) \cdot \frac{\partial L}{\partial \hat{\boldsymbol{y}}} - \hat{\boldsymbol{y}}\hat{\boldsymbol{y}}^T \cdot \frac{\partial L}{\partial \hat{\boldsymbol{y}}}$$

$$\frac{\partial L}{\partial \boldsymbol{z}} = \hat{\boldsymbol{y}} \odot \frac{\partial L}{\partial \hat{\boldsymbol{y}}} - \left(\hat{\boldsymbol{y}}^T \frac{\partial L}{\partial \hat{\boldsymbol{y}}}\right)\hat{\boldsymbol{y}}$$

Where the first transposition is required to match the dimensionalities since we are working with column vectors. Furthermore, the last line follows from simplifications of the diagonal matrix multiplied by a vector reducing to an element-wise scale of the upstream partial derivative and the second line results from the dot product reducing the vector multiplication to a scalar multiplication. Note that $\left(\text{diag}(\hat{\boldsymbol{y}}) - \hat{\boldsymbol{y}}\hat{\boldsymbol{y}}^T\right)^T = \text{diag}(\hat{\boldsymbol{y}}) - \hat{\boldsymbol{y}}\hat{\boldsymbol{y}}^T$ due to symmetry.

2.3.b Linear Layer

The object `LinearLayer2D` performs the following operation on an input vector $\boldsymbol{v}$:

$$\boldsymbol{z} = \boldsymbol{v}^T\boldsymbol{W} + \boldsymbol{b}$$
$$\hat{\boldsymbol{y}} = \boldsymbol{\sigma}(\boldsymbol{z})$$

Where $\boldsymbol{b}$ is the bias term, $\boldsymbol{W}$ is the weights, and $\boldsymbol{\sigma}$ is the activation function, $\boldsymbol{z}$ is just an intermediary step used for convenience.

We have, however, modified this operation slightly so that we can remove the separate bias term and include it directly in an expanded weight matrix $\boldsymbol{W_b}$. This is achieved by adding a row filled with ones to $\boldsymbol{W}$. So the operation actually becomes:

$$\boldsymbol{z} = \boldsymbol{v}^T\boldsymbol{W_b}$$
$$\hat{\boldsymbol{y}} = \boldsymbol{\sigma}(\boldsymbol{z})$$

The gradients are calculated as following during the backward pass:

$\hat{\boldsymbol{y}}$ w.r.t the input vector $\boldsymbol{v}$

$$z_i = v_1 \boldsymbol{W_b}_{1i} + v_2 \boldsymbol{W_b}_{2i} + ... + v_i \boldsymbol{W_b}_{ii} + ... + v_N \boldsymbol{W_b}_{ni}$$

$$\frac{\partial \boldsymbol{z_i}}{\partial \boldsymbol{v_j}} = \boldsymbol{W_b}_{ji}$$

$$\frac{\partial \boldsymbol{z}}{\partial \boldsymbol{v}} = \begin{pmatrix} \frac{\partial z_1}{\partial v_1} & \frac{\partial z_1}{\partial v_2} & \cdots & \frac{\partial z_1}{\partial v_N} \\ \frac{\partial z_2}{\partial v_1} & \frac{\partial z_2}{\partial v_2} & \cdots & \frac{\partial z_2}{\partial v_N} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial z_M}{\partial v_1} & \frac{\partial z_M}{\partial v_2} & \cdots & \frac{\partial z_M}{\partial v_N} \end{pmatrix} = \boldsymbol{W_b}^T$$

$$\frac{\partial \hat{\boldsymbol{y}}}{\partial \boldsymbol{z}} \text{ is taken from the activation function with } \frac{\partial L}{\partial \hat{\boldsymbol{y}}} \text{ applied}$$

$$\frac{\partial \hat{\boldsymbol{y}}}{\partial \boldsymbol{v}} = \frac{\partial \hat{\boldsymbol{y}}}{\partial \boldsymbol{z}} \cdot \boldsymbol{W_b}^T$$

Therefore, we can calculate $\frac{\partial L}{\partial \boldsymbol{v}}$ with:

$$\frac{\partial L}{\partial \boldsymbol{v}} = \left(\frac{\partial \hat{\boldsymbol{y}}}{\partial \boldsymbol{v}}\right)^T \cdot \frac{\partial L}{\partial \hat{\boldsymbol{y}}}$$

$$\frac{\partial L}{\partial \boldsymbol{v}} = \left(\left(\boldsymbol{W_b}^T\right)^T \cdot \left(\frac{\partial \hat{\boldsymbol{y}}}{\partial \boldsymbol{z}}\right)^T\right) \cdot \frac{\partial L}{\partial \hat{\boldsymbol{y}}}$$

$$\frac{\partial L}{\partial \boldsymbol{v}} = \boldsymbol{W_b} \cdot \left(\frac{\partial \hat{\boldsymbol{y}}}{\partial \boldsymbol{z}}\right)^T \cdot \frac{\partial L}{\partial \hat{\boldsymbol{y}}}$$

$$\frac{\partial L}{\partial \boldsymbol{v}} = \boldsymbol{W_b} \cdot \frac{\partial L}{\partial \boldsymbol{z}}$$

$\hat{\boldsymbol{y}}$ w.r.t the weight matrix $\boldsymbol{W_b}$

$$z_i = v_1 \boldsymbol{W_b}_{1i} + v_2 \boldsymbol{W_b}_{2i} + ... + v_i \boldsymbol{W_b}_{ii} + ... + v_N \boldsymbol{W_b}_{ni} + 1 \cdot \boldsymbol{W_b}_{(n+1)i}$$

$$\frac{\partial z_i}{\partial \boldsymbol{W_b}_{kj}} = \begin{cases} v_k \text{ if } i = j \\ 0 \text{ if } i \neq j \end{cases}$$

$$\frac{\partial \boldsymbol{z}}{\partial \boldsymbol{W_b}} = \left[\begin{pmatrix} v_1 & 0 & ... & 0 \\ v_2 & 0 & ... & 0 \\ \vdots & \vdots & \ddots & \vdots \\ v_N & 0 & ... & 0 \\ 1 & 0 & ... & 0 \end{pmatrix} \middle| \begin{pmatrix} 0 & v_1 & ... & 0 \\ 0 & v_2 & ... & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & v_N & ... & 0 \\ 0 & 1 & ... & 0 \end{pmatrix} \middle| ... \middle| \begin{pmatrix} 0 & 0 & ... & v_1 \\ 0 & 0 & ... & v_2 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & ... & v_N \\ 0 & 0 & ... & 1 \end{pmatrix} \right]$$

Therefore, we can calculate $\frac{\partial L}{\partial \boldsymbol{W_b}}$ where $\frac{\partial L}{\partial \boldsymbol{z}}$ is our upstream gradient. We can do this by applying the chain rule element-wise:

$$\frac{\partial L}{\partial \boldsymbol{W_b}_{ij}} = \frac{\partial L}{\partial z_j} \cdot \frac{\partial z_j}{\partial \boldsymbol{W_b}_{ij}} = \frac{\partial L}{\partial z_j} \cdot v_{b,i}$$

This is equivalent to the outer product:

$$\frac{\partial L}{\partial \boldsymbol{W_b}} = \boldsymbol{v_b}\left(\frac{\partial L}{\partial \boldsymbol{z}}\right)^T$$

$$\frac{\partial L}{\partial \boldsymbol{W_b}} = \begin{pmatrix} v_1 \\ v_2 \\ \vdots \\ v_N \\ 1 \end{pmatrix} \begin{pmatrix} \frac{\partial L}{\partial z_1} & \frac{\partial L}{\partial z_2} & \cdots & \frac{\partial L}{\partial z_M} \end{pmatrix}$$

$$\frac{\partial L}{\partial \boldsymbol{W_b}} = \begin{pmatrix} v_1\frac{\partial L}{\partial z_1} & v_1\frac{\partial L}{\partial z_2} & \cdots & v_1\frac{\partial L}{\partial z_M} \\ v_2\frac{\partial L}{\partial z_1} & v_2\frac{\partial L}{\partial z_2} & \cdots & v_2\frac{\partial L}{\partial z_M} \\ \vdots & \vdots & \ddots & \vdots \\ v_N\frac{\partial L}{\partial z_1} & v_N\frac{\partial L}{\partial z_2} & \cdots & v_N\frac{\partial L}{\partial z_M} \\ \frac{\partial L}{\partial z_1} & \frac{\partial L}{\partial z_2} & \cdots & \frac{\partial L}{\partial z_M} \end{pmatrix}$$

$$\frac{\partial L}{\partial \boldsymbol{W_b}} = \boldsymbol{v_b} \otimes \frac{\partial L}{\partial \boldsymbol{z}}$$

2.3.c Averaging Layer:

The `LinearLayerAvgMixer` averages context embeddings by:

1. Encoding each context word index as a one-hot vector $e_i \in \mathbb{R}^V$
2. Multiplying by the weight matrix: $e_i W = W_{[i]}$ (row i of W)
3. Averaging the results

Step 1 and 2 together are equivalent to a direct row lookup, since multiplying a one-hot vector by a matrix simply selects the corresponding row:

$$\frac{1}{n}\sum_{k=1}^{n} e_{i_k} W = \frac{1}{n}\sum_{k=1}^{n} W_{[i_k]}$$

`EmbeddingAverageLayer` implements this directly without large matrix multiplication and memory costly allocations. It is a mathematically equivalent simplification used to speed up training.

2.4 Loss

2.4.a Binary Cross Entropy Loss

The Binary cross entropy loss function is a loss function that is primarily used to compare probability vectors (vectors where elements sum to 1) against binary vectors (where a single element is set to 1 and there rest are set to 0.) This is usually used in binary classification where a model predicts a probability of a sample having some label, for example, cat, dog, fish. In binary classification there is only a single correct label but to improve model performance we allow it to be kind of uncertain, so it can predict 90% dog, 5% cat, 5% fish.

The loss is given by:

$$L = -y\log(\hat{y}) + (1-y)\log(1-\hat{y})$$

2.4.b Negative Sampling Loss

Instead of computing softmax over the full vocabulary, negative sampling reformulates the problem as binary classification. For a context vector $\boldsymbol{h}$ and target word index p, the loss is:

$$L = -\log \sigma(\boldsymbol{h} \cdot \boldsymbol{w}_p) - \sum_{i=1}^{k} \log\left(1 - \sigma\left(\boldsymbol{h} \cdot \boldsymbol{w}_{n_i}\right)\right)$$

Where $\boldsymbol{w}_p$ is the output embedding of the positive (target) word, $\boldsymbol{w}_{n_i}$ are the output embeddings of k randomly sampled negative words, and σ is the sigmoid function.

The gradients are:

$$\frac{\partial L}{\partial \boldsymbol{w}_p} = (\sigma(\boldsymbol{h} \cdot \boldsymbol{w}_p) - 1) \cdot \boldsymbol{h}$$

$$\frac{\partial L}{\partial \boldsymbol{w}_{n_i}} = \sigma\left(\boldsymbol{h} \cdot \boldsymbol{w}_{n_i}\right) \cdot \boldsymbol{h}$$

$$\frac{\partial L}{\partial \boldsymbol{h}} = (\sigma(\boldsymbol{h} \cdot \boldsymbol{w}_p) - 1) \cdot \boldsymbol{w}_p + \sum_{i=1}^{k} \sigma\left(\boldsymbol{h} \cdot \boldsymbol{w}_{n_i}\right) \cdot \boldsymbol{w}_{n_i}$$

Negatives are sampled from a smoothed unigram distribution $P(w) \propto f(w)^{0.75}$ where $f(w)$ is the word frequency. The smoothing reduces the dominance of very frequent words.

This loss method was implemented to improve the efficiency of training and was discussed by Google in their implementation.

2.5 Training

Training is performed using mini-batch stochastic gradient descent (SGD). Each epoch iterates over the full training set in batches of 32 context-target pairs. Within each batch, gradients are accumulated sample-by-sample and then applied once as an averaged update at the end of the batch.

Two separate training loops were implemented: train for the softmax CBOW model and train_ns for the negative sampling models. The reason for this split is that the NegativeSamplingLoss object maintains its own trainable output embeddings which sit outside the Model object. This means its weight update cannot be handled by the model's backpropagate method and must instead be called explicitly inside the training loop. Combining these into a single universal training loop is a part of potential future work on the project.

Both loops track training and validation loss per epoch. The validation pass runs forward passes over the held-out 20% of sentences without accumulating or applying any gradients, using clear_caches to discard the cached forward-pass inputs that would otherwise accumulate unused.

The final optimized models were trained with the following hyperparameters, kept fixed across all three corpus sizes so that only data volume varied between them:

Hyperparameter	**Value**
Embedding dimension	50
Negative samples k	5
Learning rate	0.2
Batch size	32
Epochs	10
Train / val split	80% / 20%

Table 1: Final hyperparameters for model training and evaluation

The learning rate of 0.2 is relatively high compared to typical neural network deep learning settings, but is appropriate here since the model is shallow and the gradients are sparse resulting in each

update only touching the embedding rows corresponding to the context words and sampled negatives as opposed to the usual full weight matrix.

Model Evaluation

3.1 Visualization methods

3.1.a PCA:

PCA is one of the oldest tricks in the book when it comes to reducing dimensionality of data in order to make it possible to visualize in a 2d, and sometimes 3d, space. Its foundational paper was released all the way back in 1933 by a Harald Hotelling, making it almost 100 years old.

PCA relies on two important concepts from the field of linear algebra, eigenvectors and matrix transformations (base changes to be more exact). The PCA method works in several steps:

- Initially the eigenvectors of the data's covariance matrix are computed. Each eigenvector $\boldsymbol{\lambda}_i$ defines a direction in the feature space, and its corresponding eigenvalue measures how much of the total variance in the data is explained by that direction. For a dataset with three features the covariance matrix and its eigenvectors look like:

$$\begin{pmatrix} 1 & \sigma_{12}^2 & \sigma_{13}^2 \\ \sigma_{12}^2 & 1 & \sigma_{23}^2 \\ \sigma_{13}^2 & \sigma_{23}^2 & 1 \end{pmatrix} \rightarrow \boldsymbol{\lambda}_1 = \begin{pmatrix} \lambda_{11} \\ \lambda_{21} \\ \lambda_{31} \end{pmatrix}, \boldsymbol{\lambda}_2 = \begin{pmatrix} \lambda_{12} \\ \lambda_{22} \\ \lambda_{32} \end{pmatrix}, \boldsymbol{\lambda}_3 = \begin{pmatrix} \lambda_{13} \\ \lambda_{23} \\ \lambda_{33} \end{pmatrix}$$

- The eigenvectors are then sorted by their eigenvalues in descending order, so that the first principal component $\boldsymbol{\lambda}_1$ captures the direction of greatest variance, the second the next greatest, and so on.
- Finally, the top k eigenvectors are selected and used to form a projection matrix $\boldsymbol{P} \in \mathbb{R}^{n \times k}$. The original data is projected into this lower-dimensional subspace via $\boldsymbol{Z} = \boldsymbol{X}\boldsymbol{P}$, producing the reduced representation used for visualisation.

A key limitation of PCA is that it is a purely linear method, meaning it can only capture structure that lies along straight axes in the original space. Non-linear relationships between words, such as semantic clusters that form curved or folded surfaces in embedding space, are invisible to PCA and may be better revealed by UMAP below.

3.1.b UMAP:

UMAP (Uniform Manifold Approximation and Projection) is a considerably more recent method, introduced by McInnes, Healy and Melville in 2018. Unlike PCA, which is a linear technique restricted to capturing variance along orthogonal axes, UMAP is a non-linear dimensionality reduction method built on the theory of Riemannian geometry and algebraic topology.

The core idea is that high-dimensional data is assumed to lie on a low-dimensional manifold embedded in the ambient space. UMAP constructs a weighted graph representing the local neighborhood structure of the data, where the weight of the edge between points x_i and x_j is given by a fuzzy membership strength:

$$\overline{p}_{ij} = p_{ij} + p_{ji} - p_{ij} \cdot p_{ji}, \quad p_{ij} = \exp\left(-\frac{d(x_i, x_j) - \rho_i}{\sigma_i}\right)$$

where ρ_i is the distance from x_i to its nearest neighbor and σ_i is a per-point bandwidth chosen so that the total weight out of x_i equals $\log_2(k)$ for the chosen number of neighbors k. A

corresponding low-dimensional graph is then optimised to have a similar fuzzy structure, minimising the cross-entropy between the two representations:

$$\mathcal{L} = -\sum_{i \neq j} \left[\overline{p}_{ij} \log q_{ij} + \left(1 - \overline{p}_{ij}\right) \log\left(1 - q_{ij}\right) \right]$$

Two hyperparameters control the resulting layout. `n_neighbors` affects how much local versus global structure is preserved where small values emphasise tight local clusters while larger values give a broader picture of the overall topology of the data. `min_dist` controls how tightly points can be packed together in the embedding; a smaller value produces more compact clusters at the cost of less faithful representation of inter-cluster distances. The cosine metric is used here rather than Euclidean distance, which is more appropriate for word embedding vectors where the direction of the vector encodes meaning rather than its magnitude.

3.2 Results

3.2.a Training Curves

The training and validation loss curves across all three corpus sizes are shown in Figure 1. All three models show consistent decreasing loss across epochs, however, they differ in levels of overfitting where the small corpus model shows signs of potential future improvement. Furthermore, the medium corpus size begins overfitting after epoch 4 while the large corpus model continues to slowly improve with consistent generalization gap. These differences likely stem from the vocabulary size affecting how diluted the gradient update becomes for each sample of a context-target pair. Higher vocabulary size meaning a single sample causes less of an impact on the total gradient for the batch. Nevertheless, as expected, the larger corpus models converge to lower absolute loss values, reflecting that more training data provides richer co-occurrence for the embeddings to learn from.

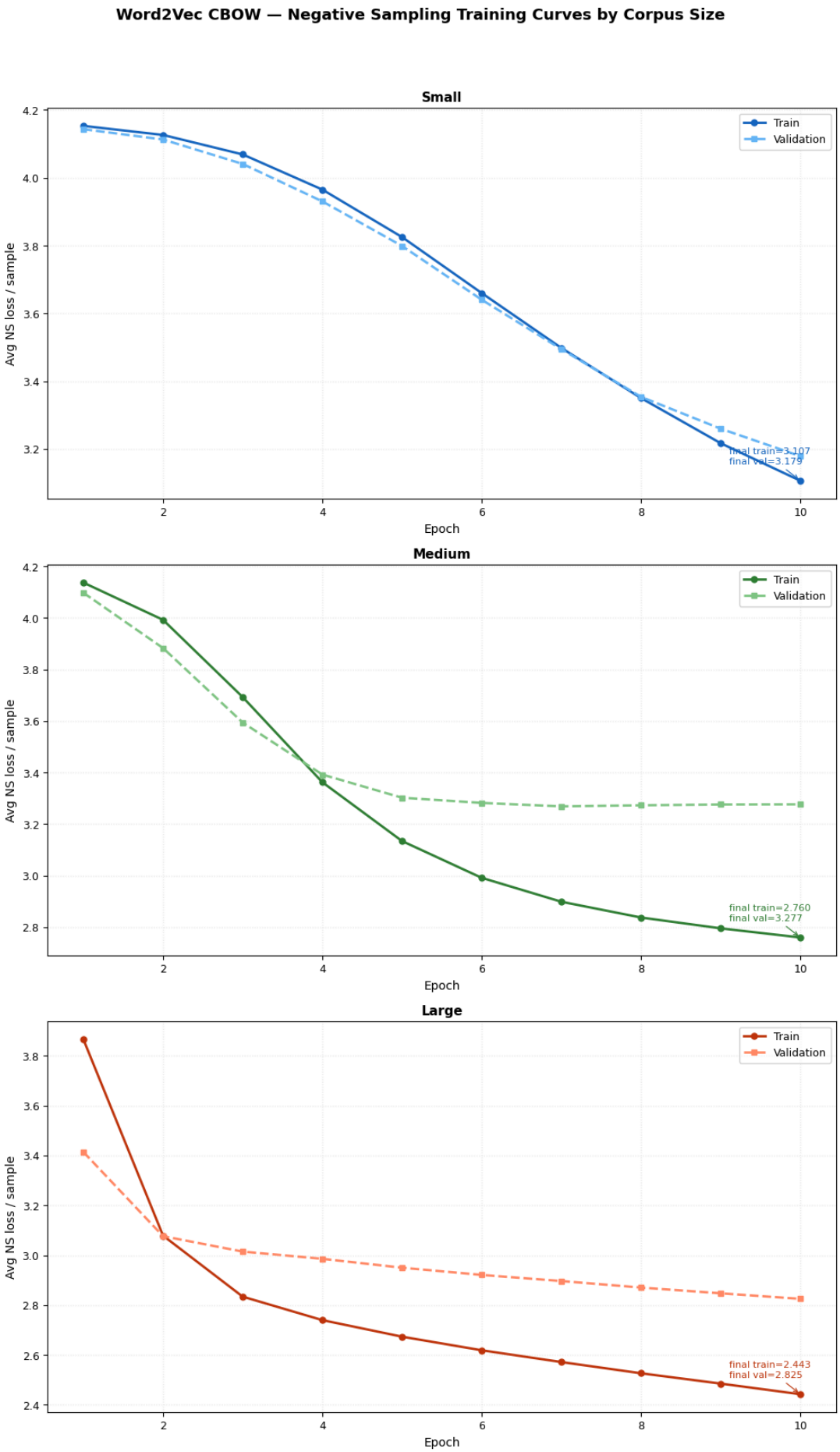

Figure 1: Negative sampling training and validation loss per epoch across Small, Medium, and Large corpus sizes.

3.2.b Nearest Neighbor Analysis

To evaluate the quality of the learned embeddings qualitatively, we query the k nearest neighbors of a set of probe words by cosine similarity. The results for the large corpus model are shown in Table 2, and a cross-scale comparison for a selection of probe words is shown in Table 3.

The cross-scale comparison reveals a clear progression in embedding quality as corpus size increases. On the small corpus (200 sentences), the vocabulary is severely limited with for example "king" being absent entirely. The nearest neighbors seem to be represented by noise, dominated by stop words and random co-occurrences with no discernible semantic pattern. On the medium corpus (1000 sentences), cosine similarities are higher but neighbors are still predominantly function words such as "in", "for", "and", reflecting that the model has learned syntactic context more than semantic meaning. On the large corpus (5000 sentences), the results improve considerably. For example, the nearest neighbors of "first" include "second", "best", and "new" which are words with directly similar meaning where "second" naturally comes after "first" and "best" as well as "new" showing learned understanding of the meaning of primality and uniqueness of being first. Similarly, "world" retrieves words like "game", "league", and "american", which is plausible to frequently co-occur in a Wikipedia-style text of the source corpus. Here it is clear that our choice to not implement a shuffle in the dataloader may be severely affecting the results where articles using words in specific contexts can heavily skew the results.

While it has learned some plausible representation of words, the large corpus results still fall short of what a production Word2Vec model would produce. The neighbors of "king" include "project", "season", and "rock" which are clearly semantically unrelated words. Furthermore, "war" retrieves "season" as its closest neighbor, which may be an indication of co-occurence in topics such as sports in the corpus rather than genuine semantic understanding. The high baseline cosine similarities across all queries (most neighbors scoring above 0.85) suggest the embedding space has not fully separated into distinct semantic regions, with many words still clustered too closely together. This is consistent with the relatively small corpus size and limited training epochs compared to the original Word2Vec paper, which trained on a corpus of roughly 100 billion words.

Query	#1	#2	#3	#4	#5	#6	#7	#8
time	british (0.906)	day (0.893)	same (0.887)	team (0.883)	first (0.881)	end (0.870)	north (0.870)	new (0.867)
year	only (0.885)	but (0.877)	time (0.856)	new (0.853)	egyptian (0.853)	british (0.850)	made (0.848)	half (0.847)
century	following (0.860)	before (0.854)	under (0.845)	at (0.840)	during (0.829)	against (0.828)	from (0.825)	team (0.823)
age	archaeology (0.741)	best (0.728)	ga (0.727)	number (0.722)	four (0.721)	university (0.721)	out (0.717)	townsend (0.716)
world	game (0.872)	british (0.853)	second (0.850)	early (0.833)	league (0.830)	american (0.827)	top (0.825)	school (0.822)
city	first (0.888)	top (0.877)	team (0.864)	end (0.857)	second (0.851)	british (0.851)	site (0.851)	season (0.848)
war	season (0.870)	new (0.866)	british (0.863)	first (0.860)	all (0.849)	end (0.849)	team (0.849)	state (0.846)
government	team (0.856)	gods (0.839)	song (0.836)	army (0.834)	site (0.830)	film (0.826)	fleet (0.826)	album (0.825)
american	game (0.896)	season (0.878)	league (0.875)	only (0.867)	british (0.867)	national (0.865)	three (0.863)	north (0.860)
people	an (0.876)	out (0.871)	many (0.865)	some (0.859)	their (0.856)	these (0.854)	each (0.851)	all (0.851)
king	project (0.827)	season (0.753)	team (0.753)	rock (0.752)	building (0.750)	area (0.750)	although (0.747)	before (0.744)
queen	player (0.641)	aviation (0.576)	rays (0.562)	as (0.560)	for (0.553)	full (0.549)	association (0.546)	or (0.541)
church	film (0.856)	album (0.826)	ship (0.819)	second (0.817)	project (0.811)	team (0.807)	song (0.807)	last (0.806)
new	second (0.912)	first (0.908)	season (0.893)	british (0.891)	only (0.890)	best (0.887)	two (0.885)	all (0.883)
great	side (0.784)	third (0.777)	made (0.773)	main (0.767)	club (0.763)	season (0.761)	album (0.758)	school (0.755)
first	second (0.936)	season (0.917)	new (0.908)	team (0.904)	best (0.903)	film (0.896)	york (0.895)	all (0.894)
history	some (0.879)	first (0.877)	games (0.868)	north (0.865)	its (0.863)	all (0.861)	their (0.857)	new (0.852)

Table 2: K-Nearest neighbor results for probe words on large model with $k = 8$

Query	Model	Top-5 neighbors (Cosine Similarity)
time	Small	mary (0.398), available (0.391), over (0.371), major (0.354), removed (0.353)
	Medium	in (0.817), at (0.791), and (0.782), was (0.782), <PAD> (0.780)
	Large	british (0.906), day (0.893), same (0.887), team (0.883), first (0.881)
world	Small	waldrons (0.415), 1911 (0.410), has (0.404), such (0.399), form (0.395)
	Medium	gods (0.749), game (0.748), of (0.746), while (0.738), for (0.735)
	Large	game (0.872), british (0.853), second (0.850), early (0.833), league (0.830)
war	Small	the (0.510), were (0.503), and (0.497), was (0.493), rock (0.473)
	Medium	between (0.643), only (0.625), may (0.624), was (0.615), with (0.609)
	Large	season (0.870), new (0.866), british (0.863), first (0.860), all (0.849)
king	Small	*word not in vocabulary*
	Medium	blackie (0.525), during (0.522), they (0.498), to (0.496), has (0.488)
	Large	project (0.827), season (0.753), team (0.753), rock (0.752), building (0.750)
first	Small	once (0.488), by (0.447), unk (0.432), that (0.431), an (0.422)
	Medium	for (0.859), with (0.833), and (0.832), is (0.831), in (0.828)
	Large	second (0.936), season (0.917), new (0.908), team (0.904), best (0.903)

Table 3: K-Nearest neighbor results for probe words on all three model with $k = 5$

3.2.c PCA Visualization

Figure 2 shows a PCA projection of the 150 most frequent words from the large corpus embedding. Points are colored by log word frequency. The projection captures only a modest fraction of the total

variance across the 50-dimensional embedding space, which is an expected result. It follows from PCA's linear projection not being able to fully represent the non-linear structure of a learned embedding space in only two dimensions.

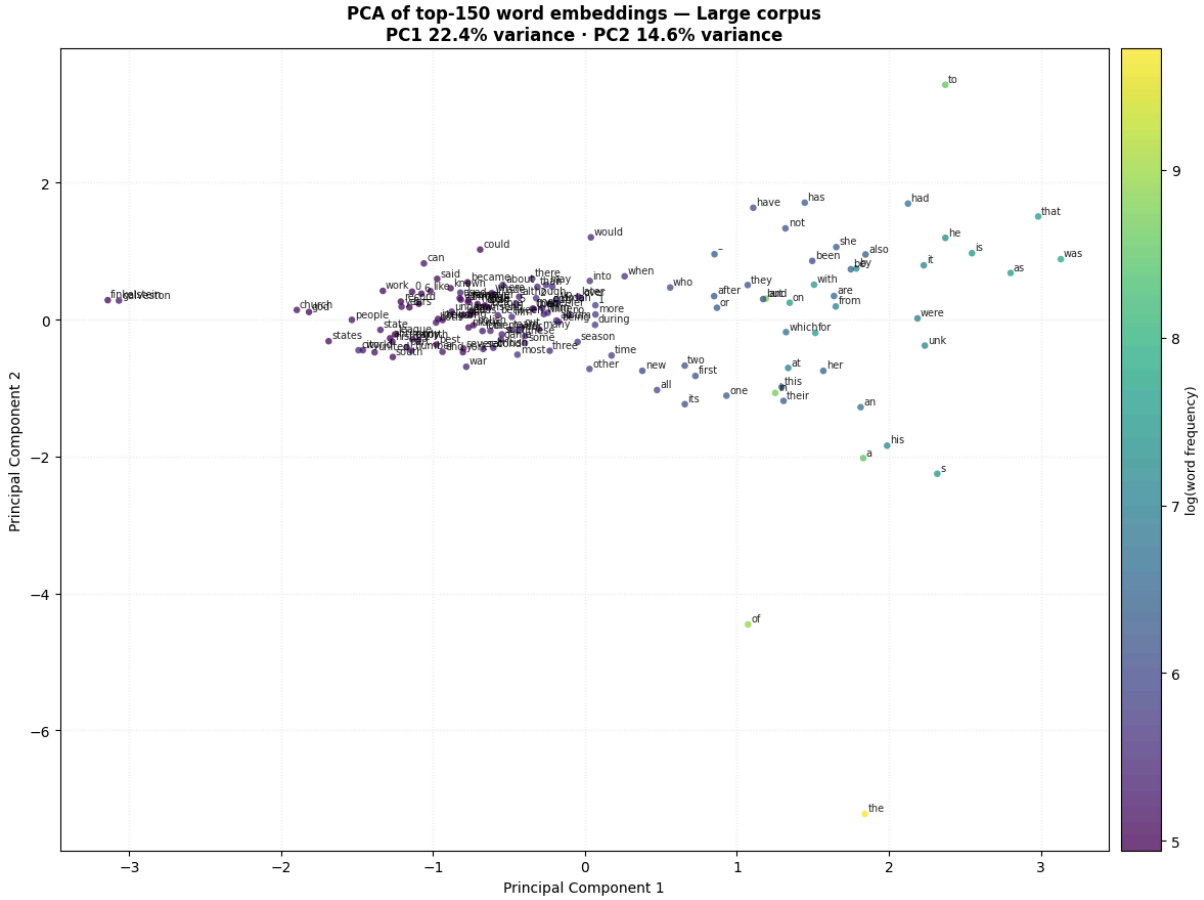

Figure 2: PCA projection of the 150 most frequent word embeddings from the large corpus model. Color indicates log word frequency.

3.2.d UMAP Visualization

Figure 3 shows the same embeddings projected using UMAP with cosine distance, $k = 15$ neighbors and min_dist $= 0.1$. UMAP is better suited to this task than PCA since it preserves local neighborhood structure and can represent non-linear relationships in the embedding space. This superiority over PCA for word embedding representations is clear even in our extremely limited corpus where we observe that numerical words of numbers are tightly clustered in their own space away from the other words. However words like "four" and "number" are not recognized as being numerical in the same way. Nevertheless, local word clusters exist that make sense. Such as "team", "world", "game", and "league" occurring in a cluster at approximately (4.5, −1)

Figure 3: UMAP projection of the 150 most frequent word embeddings from the large corpus model using cosine distance.

Compared to the PCA plot, the UMAP projection produces slightly more separated local clusters, suggesting that the embeddings do contain local neighborhood structure that PCA flattens out. However, the overall layout remains relatively diffuse without any sharply defined semantic groupings besides the number words. This is consistent with our previous observations from k-nearest neighbors of a model that has learned partial but not complete semantic structure given the corpus size.

Finally a PCA comparison plot for corpus sizes is given in Figure 4.

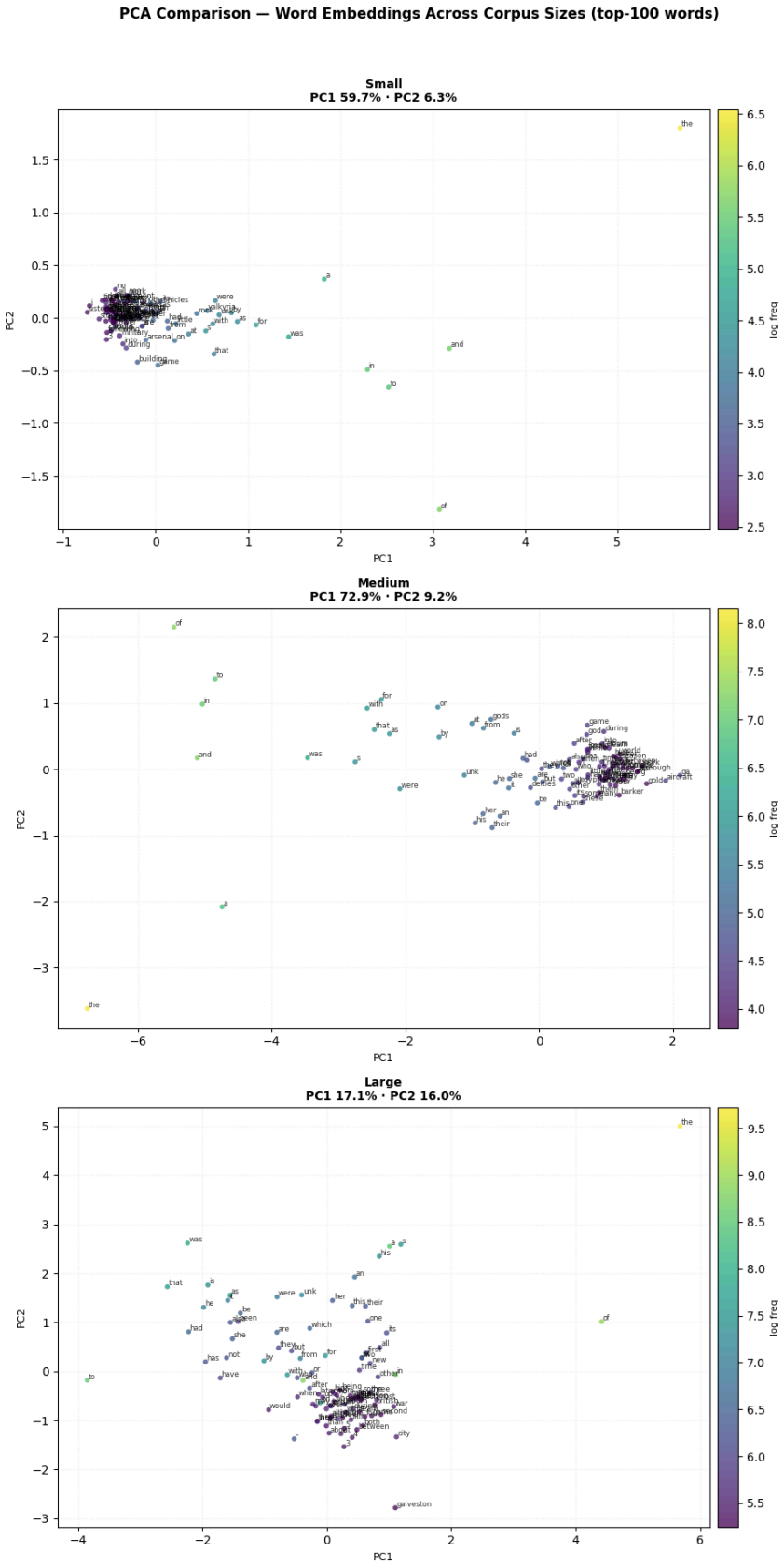

Figure 4: PCA projection of the 100 most frequent word embeddings from each corpus model. Color indicates log word frequency.

Conclusion

This project set out to replicate the core Word2Vec pipeline from scratch using only PyTorch for tensor operations, and to study how corpus size and output objective affect the quality of the learned embeddings.

The implementation successfully reproduces the key components described in the original 2013 paper which includes a tokenizer with preprocessing, a CBOW context-target pair generator, a manually implemented embedding layer with mini-batch gradient descent, and a negative sampling loss with smoothed unigram noise distribution. Three model variants were compared: softmax CBOW, unoptimized negative sampling CBOW, and an optimized embedding lookup model which reveals an interesting practical finding. Theoretical algorithmic complexity does not always translate directly to empirical runtime. The unoptimized negative sampling model was slower than the softmax model at small vocabulary sizes, because the Python loop overhead of individual sigmoid operations outweighed the savings from avoiding the full softmax normalization. The optimized `EmbeddingAverageLayer`, which replaces the dense one-hot matrix multiply with a direct row lookup, was required to achieve the required speedup at scale.

The nearest neighbor analysis and visualizations confirm that the large corpus model has learned meaningful embedding structure to some degree. Words like “first” and “second” cluster together, and contextually co-occurring words appear as neighbors with high cosine similarity. The UMAP visualization further revealed that numerical words form a tight isolated cluster in the embedding space, demonstrating that the model has identified at least one coherent semantic category without any explicit supervision. However, the results also make clear that 5000 sentences is far from sufficient to produce the kind of clean semantic geometry that the original Word2Vec model demonstrated. The embedding space has learned co-occurrence patterns but has not yet fully separated into distinct semantic regions, evidenced by the uniformly high cosine similarities across probe words and semantically unrelated nearest neighbors for words like “king” and “war”. The lack of shuffling in the dataloader is likely a contributing factor, as the sequential ordering of Wikipedia articles means that topic-specific co-occurrence patterns can dominate the learned representations.

The primary limiting factor is corpus size. The original Word2Vec was trained on approximately 100 billion words while our largest model used roughly 5000 sentences from WikiText. Scaling further is the most direct path to better embeddings, but doing so on CPU with a pure Python training loop is prohibitively slow. GPU parallelization of the embedding lookup and gradient accumulation is therefore the most important direction for future work. Implementing shuffling in the streaming dataloader and extending the implementation to support Skip-gram in addition to CBOW, which was planned in the original proposal but not completed due to time constraints, are also natural next steps.

In relation to the course, this project demonstrates concretely how a latent factor model extends beyond recommendation systems into the domain of language. The embedding matrices play the same conceptual role as the latent user and item factor matrices in collaborative filtering. They are both learned representations that compress a high-dimensional co-occurrence structure into a dense low-dimensional space where proximity encodes relatedness. Building the optimization machinery from scratch gave us direct insight into how these representations form and why design choices such as the loss function, corpus size, and learning rate have the effects they do, which would have been opaque had we simply called a packaged Word2Vec implementation.